ترب بلیت

R&D PRODUCT PROPOSAL
CHALLENGES & OPPORTUNITIES

Presented by
MM Khojasteh



ترب در واقع یکی از موفق‌ترین نمونه‌های اجرای «تئوری تجمیع‌سازی» (Aggregation Theory) در اکوسیستم دیجیتال ایران است.

هدف نهایی ترب این است که به «نقطه شروع هر جستجوی خریدی در ایران» تبدیل شود.
همانطور که آمازون در آمریکا کاری کرد که مردم به جای جستجوی کالا در گوگل، مستقیماً وارد آمازون شوند؛
ایده نهایی ترب این است که به دروازه ورود (Gateway) و موتور جستجوی دیفالت تمام خریداران ایرانی تبدیل شود.



دیدگاه پروداکتی (Product Vision)

- **موتور جستجو بودن، نه فروشگاه (Search vs. E-commerce):** ترب خودش را درگیر انبارداری، لجستیک، تامین کالا و پشتیبانی پس از فروش نمی‌کند. محصول آن‌ها یک «لایه نرم‌افزاری» است که روی کل بازار خرده‌فروشی ایران کشیده شده است.

- **دموکراتایز کردن بازار (Democratization of Commerce):** دیدگاه آن‌ها این است که یک فروشگاه کوچک در یک شهرستان دورافتاده، اگر قیمت و خدمات بهتری ارائه دهد، باید بتواند دقیقا در کنار غول‌هایی مثل دیجی‌کالا یا تکنولایف نمایش داده شود و حتی بالاتر از آن‌ها قرار گیرد.

- **کاهش اصطکاک (Frictionless UX):** رابط کاربری ترب به شدت مینیمال، سریع و صرفاً مبتنی بر «جستجو» طراحی شده است. هیچ محتوای اضافه‌ای که کاربر را از مسیر "جستجو -> مقایسه -> کلیک روی فروشگاه" منحرف کند، در این محصول وجود ندارد.

چه ارزشی خلق می‌کنند؟ (Value Proposition)

الف) برای کاربران (B2C):

- **شفافیت محض و تقارن اطلاعاتی:** کاربر دیگر نیازی ندارد برای پیدا کردن ارزان‌ترین قیمت، ده‌ها سایت یا پیج اینستاگرامی را بگردد. ترب قیمت یک کالا را در صدها فروشگاه در یک ثانیه مقایسه می‌کند.
- **کشف پذیری (Discovery):** پیدا کردن کالاهای نایاب یا قطعاتی که در فروشگاه‌های بزرگ ناموجود هستند، در شبکه‌ای از هزاران فروشگاه کوچکِ متصل به ترب ممکن می‌شود.
- **تزریق اعتماد (Trust as a Service):** با معرفی سرویس «ضمانت ترب» (یک سیستم Escrow یا پرداخت امن)، ترب ارزش جدیدی خلق کرد: کاربر پول را به ترب می‌دهد، فروشگاه کالا را می‌فرستد و در صورت تایید کاربر، پول به حساب فروشگاه می‌رود. این یعنی غلبه بر بزرگترین مانع خرید از سایت‌های ناشناخته.

ب) برای فروشگاه‌ها (B2B):

- **جذب مشتری بدون نیاز به برندسازی و سئو:** فروشگاه‌های کوچک توان مالی رقابت در سئو با دیجی‌کالا را ندارند. ترب با مدل کلیکی (CPC) خود، ترافیک کاملاً هدفمند (کاربری که قصد قطعی خرید دارد) را به سمت این فروشگاه‌ها سرازیر می‌کند. ترب نقش بازاریاب و تیم سئوی این فروشگاه‌ها را بازی می‌کند.

چگونه بین رقبا فعالیت دارند و در بازار مانده‌اند؟ (Competitive Strategy)
ترب در بازاری کار می‌کند که دیجی‌کالا (با سهم بازار عظیم) و ایمالز (رقیب مستقیم) حضور دارند. استراتژی بقای آن‌ها بسیار هوشمندانه بوده است:

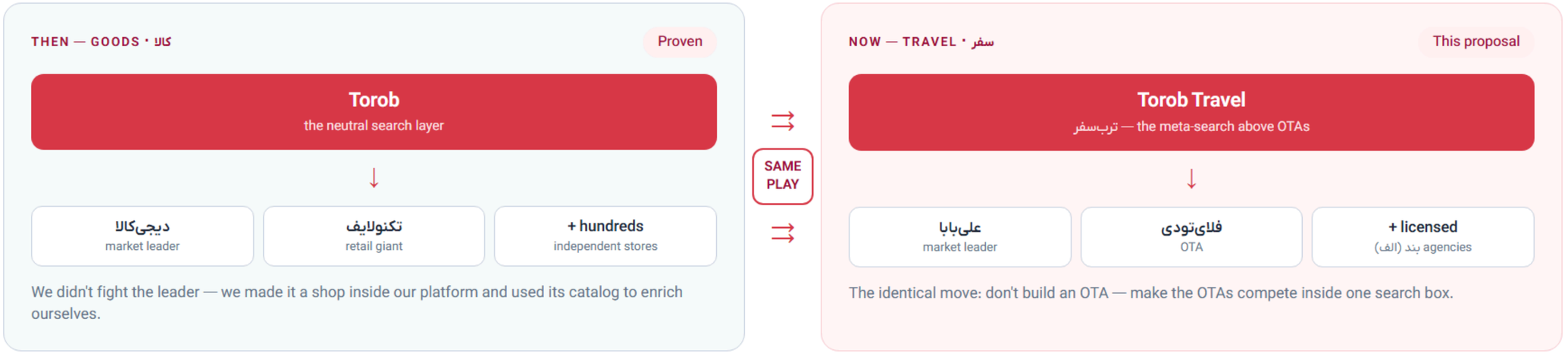
- **قرار گرفتن در یک لایه بالاتر (Super-Layer):** ترب به جای رقابت مستقیم با دیجی‌کالا، دیجی‌کالا را تبدیل به یکی از فروشگاه‌های داخل پلتفرم خود کرد! آن‌ها با این کار، به جای جنگیدن با رهبر بازار، از دیتا و دیتابیس کالایی آن برای غنی کردن پلتفرم خود استفاده کردند.
- **استفاده از ضعف رقبای بزرگ (Market Fragmentation):** هرچه کمیسیون‌های فروشگاه‌های بزرگی مثل دیجی‌کالا بیشتر شد و قیمت‌ها در آن‌ها بالا رفت، فروشندگان ترغیب شدند سایت‌های مستقل خود را بزنند. اما این سایت‌ها ترافیک نداشتند. ترب دقیقاً در همین شکاف بازار ایستاد و ترافیک را به این سایت‌های مستقل داد.
- **برتری بر ایمالز با تکنولوژی و موبایل:** ایمالز زودتر از ترب شروع کرد، اما ترب با توسعه یک اپلیکیشن موبایل به شدت سریع، الگوریتم‌های خزش (Crawling) قوی‌تر برای آپدیت لحظه‌ای قیمت‌ها، و رابط کاربری مدرن‌تر، گوی سبقت را ربود.

رجحان (Unfair Advantage) و مزیت رقابتی آن‌ها چیست؟ رجحان استراتژیک ترب در چند مورد کلیدی خلاصه می‌شود:

- **هژمونی مطلق سئو (Programmatic SEO):** ترب به ازای هر کالا، هر برند و هر دسته‌بندی، یک صفحه بهینه‌شده دارد. امروزه شما نام هر کالایی را در گوگل به فارسی سرچ کنید، ترب در ۳ لینک اول حضور دارد. آن‌ها عملاً در حال «آربیتراژ ترافیک» هستند؛ ترافیک را ارزان (یا رایگان) از گوگل می‌گیرند و به صورت کلیکی به فروشگاه‌ها می‌فروشند.
- **مقیاس‌پذیری با دارایی صفر (Asset-light Scalability):** ترب نه انباری دارد، نه پیکی، نه کالایی. وقتی تورم بالا می‌رود یا زنجیره تامین به مشکل می‌خورد، فروشگاه‌ها آسیب می‌بینند، اما ترب صرفاً ترافیک را هدایت می‌کند و هزینه کلیک را دریافت می‌کند. حاشیه سود آن‌ها به عنوان یک شرکت نرم‌افزاری بسیار بالاتر از شرکت‌های تجارت الکترونیک سنتی است.
- **اثر شبکه‌ای (Network Effect) قدرتمند:** هرچه فروشگاه‌های بیشتری به ترب بپیوندند، قیمت‌ها رقابتی‌تر و تنوع بیشتر می‌شود -> در نتیجه کاربران بیشتری به ترب می‌آیند -> در نتیجه فروشگاه‌های بیشتری مجبورند برای زنده ماندن در ترب ثبت‌نام کنند. این یک چرخه رشد بی‌نهایت است که شکستن آن برای رقبای جدید تقریبا غیرممکن است.

ماتریس ارزیابی استراتژیک شاخه‌های جدید

مدل درآمدی پیشنهادی	ارزش اصلی پیشنهادی برای کاربر	بار عملیاتی و رگولاتوری	همسویی با الگوریتم و DNA ترب	حوزه
کمیسیون به ازای فروش (CPA) / کلیک (CPC)	شفافیت کامل قیمت + هشدارهای هوشمند + خرید قسطی	پایین (صرفاً اتصال فنی API)	بسیار بالا (تطابق کامل SKU)	بلیت (سفر)
سهم از کارمزد صدور بیمه‌نامه	ارزان‌ترین حق بیمه + اقساط بدون بهره با ترب‌پی	**متوسط** (نیاز به همکاری با کارگزاران مجاز)	بالا (محصول کاملاً کالاشده)	بیمه خودرو
تبلیغات کلیکی یا فروش لید به آژانس‌های املاک	تجمیع آگهی‌ها در یک نقطه (تنها در مدل متا-سرچ)	**بسیار بالا** (تقلب گسترده، نیاز به راستی‌آزمایی)	**بسیار ضعیف** (دارایی غیرهمگن)	مسکن



اگر ترب بخواهد به بازارهایی وارد شود که غول‌هایی مثل علی‌بابا (سفر/بلیت)، اسنپ (سوپراپ و بیمه/ازکی) و جاباما/جاجیگا (اقامتگاه و ویلا) در آن حکمرانی می‌کنند، بزرگترین اشتباه استراتژیک این است که شبیه آن‌ها شود.

ترب نباید آژانس مسافرتی (OTA) یا پلتفرم مدیریت ویلا باشد؛ بلکه باید همان کاری را بکند که با دیجی‌کالا و تکنولایف کرد:
قرار گرفتن در لایه بالاتر (Super-Aggregator / Meta-Platform) و مقایسه کردنِ خودِ این غول‌ها با یکدیگر.

novelty های متمایزکننده:

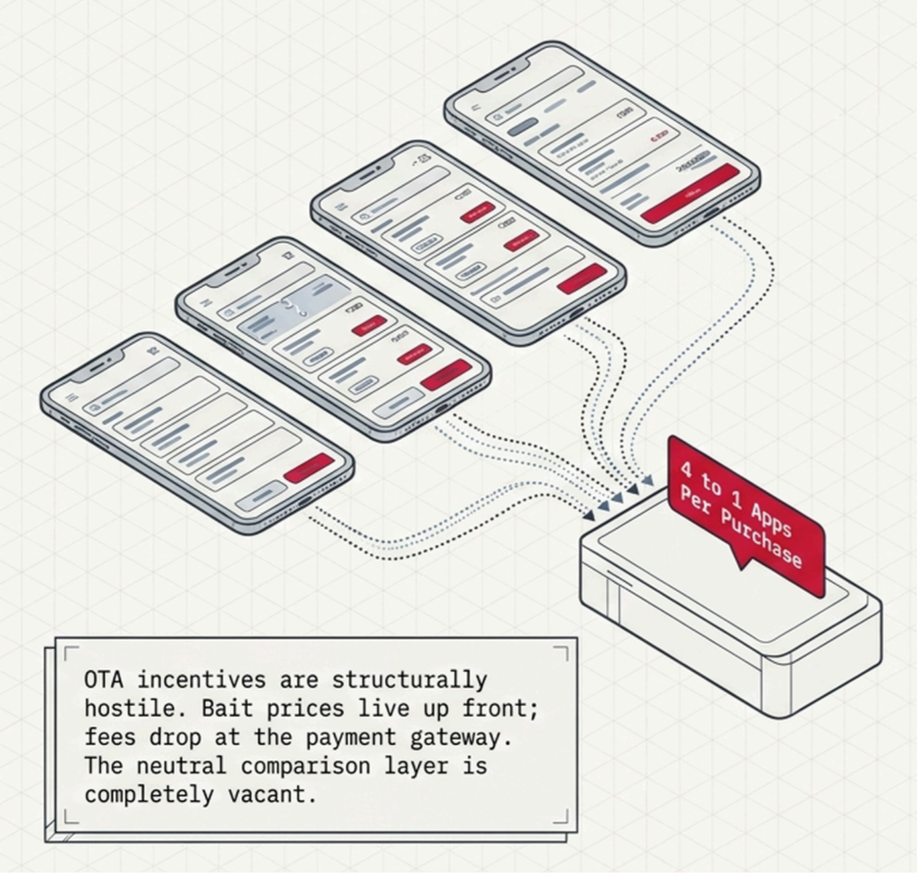
- **پیش‌بینی نوسان قیمت با هوش مصنوعی (مدل Hopper):** با تحلیل دیتای تاریخی به کاربر بگوید: «الان این بلیت را نخر؛ قیمت پرواز تهران-کیش بر اساس داده‌های ۲ ماه گذشته، معمولاً سه‌شنبه‌ها عصر ۱۵٪ ارزان‌تر می‌شود».

- **شکارچی کنسلی و پای پرواز (Seat Alert):** اعلان فوری در زمان خالی شدن صندلی‌های مسیرهای پرتقاضا (مثل مشهد در مناسبت‌ها) از تمام آژانس‌های وب.

- **شفاف‌ساز هزینه نهایی (Hidden-fee Stripper):** حذف حقه‌های مارکتینگی آژانس‌ها؛ ترب قیمت نهایی پای درگاه (شامل مالیات و کارمزد صدور) را نشان دهد نه قیمت فریبنده اولیه.



حوزه بلیت و سفر (بالاترین همسویی با ذات ترب)



این بازار بیشترین قرابت ماهیتی را با الگوریتم و هویت فعلی ترب دارد. در دنیا نیز مدل موفق Skyscanner و Google Flights نشان داده که کاربران ترجیح می‌دهند به جای باز کردن چند سایت OTA، یک متاسرچ (موتور جستجوی تجمیعی) را باز کنند.

همسویی با ذات ترب: بلیت یک کالای کاملاً استاندارد (Commodity) است؛ پرواز تهران به مشهد در یک ساعت و ایرلاین مشخص، روی ۵ آژانس مختلف (مثل علی‌بابا، فلای‌تودی، مستربلیت) دقیقاً یک محصول است، اما ممکن است با قیمت‌ها یا سهمیه‌های متفاوتی عرضه شود. این دقیقاً همان بازی Product Matching است که ترب سال‌ها در آن تبحر پیدا کرده است.

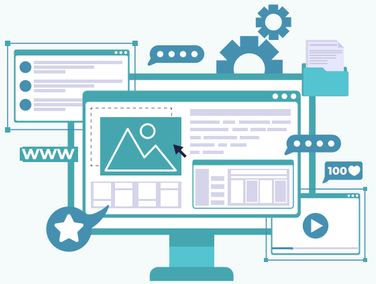
Product Sense: Why are we building this?



آنچه کاربران می‌گویند: "می‌خوام ارزون‌ترین بلیت رو پیدا کنم"

مسئله (The Real Pain): کاربر برای پیدا کردن بلیت ارزان، مجبور است ۴ اپلیکیشن (علی‌بابا، فلای‌تودی، سفرمارکت، مستربلیت) را باز کند، تاریخ‌ها را جابجا کند، و در نهایت با استرس اینکه "آیا قیمت بهتری هم هست؟" خرید کند. فرآیند کشف قیمت پر از اصطکاک و عدم شفافیت است.

فرضیه محصول: اگر ما یک موتور جستجوی تجمیعی (Aggregator) بسازیم که تنها با یک بار ورود اطلاعات، قیمت نهایی (بدون هزینه‌های پنهان) تمام آژانس‌ها را یکجا نشان دهد، کاربران جستجوی سفر خود را از ترب آغاز خواهند کرد.



استراتژی گو-تو-مارکت (GTM Strategy) و چالش‌ها

ورود آرام و هوشمندانه: ترب نباید روز اول برای کل بلیت‌ها لانچ کند. باید از پروازهای داخلی شروع کند (استانداردترین کالا)، سپس قطار، بعد اتوبوس و در نهایت پرواز خارجی.

نحوه ارتباط با آژانس‌ها (تامین‌کنندگان):

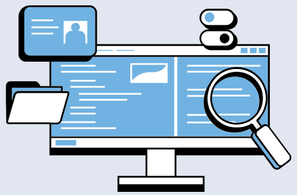
- **فاز اول (ترافیک رایگان):** ترب ابتدا پروازهای علی‌بابا، مستربلیت و ... را بدون دریافت کمیسیون (Free CPC) لیست می‌کند. ترافیک عظیمی روانه این سایت‌ها می‌شود.
- **فاز دوم (وابستگی):** آژانس‌ها به ترافیک رایگان و باکیفیت ترب معتاد می‌شوند.
- **فاز سوم (پولی‌سازی):** ترب مدل CPC یا CPA (پرداخت به ازای خرید قطعی) را روشن می‌کند. آژانسی که پول ندهد، در نتایج نمایش داده نمی‌شود و سهم بازارش را به رقبا می‌بازد. این دقیقاً همان بازی کلاسیک Aggregation Theory است.

محل قرارگیری در پروداکت فعلی (Placement & Navigation)

ترب در حال حاضر یک ساختار تک‌محوری (Single-purpose) دارد: نوار جستجوی کالا و دسته‌بندی‌ها. برای اضافه کردن بلیت، نباید این سادگی به هم بریزد.

- **مدل Tabbed Navigation (محیط ایزوله):** بهترین راهکار برای ترب، استفاده از تب‌ها (Tabs) در بالای صفحه (Header) است. درست زیر لوگوی ترب، دو تب بزرگ و واضح قرار می‌گیرد:
  a. تب پیش‌فرض: «کالا و محصولات» (همان ترب فعلی)
  b. تب جدید: «سفر و بلیت» (محیط جدید)

- چرا این مدل؟ زیرا ماهیت جستجوی کالا (سرچ باکس متنی: "گوشی سامسونگ") با ماهیت جستجوی سفر (مبدا، مقصد، تاریخ رفت‌وبرگشت) کاملاً متفاوت است. با سوئیچ کردن به تب «سفر و بلیت»، کل رابط کاربریِ زیرِ تب تغییر می‌کند و به جای نوار جستجوی متنی، ویجت جستجوی سفر (Search Widget) با فیلدهای تخصصی (مبدا، مقصد، تاریخ، تعداد مسافر) ظاهر می‌شود.





برای هندل کردن صدها هزار درخواست جستجوی همزمان (Concurrency) و آپدیت لحظه‌ای قیمت‌ها از ده‌ها آژانس،
معماری یکپارچه (Monolith) و دیتابیس‌های سنتی (مثل MySQL) به سرعت به باتلاق تبدیل می‌شوند.
اینجا دقیقاً نقطه درخشش آپاچی کافکا (Apache Kafka) است.

منطق استفاده: کافکا به عنوان یک Event Streaming Platform سه مشکل اساسی ما را حل می‌کند:

- **Decoupling (جداسازی):** سرویسی که دیتا را از علی‌بابا می‌گیرد (Scraper) نباید منتظر بماند تا دیتابیس آپدیت شود. او دیتا را در کافکا پرتاب می‌کند و می‌رود سراغ ریکوئست بعدی.
- **Buffer (ضربه گیر):** در زمان‌های پیک (مثل شب‌های تعطیلات)، ترافیک ورودی دیتابیس را زمین نمی‌زند، بلکه در صف کافکا تلنبار می‌شود و سرویس‌ها با سرعت توان خودشان آن را پردازش می‌کنند.
- **Real-time Pipelines:** می‌توانیم چندین Consumer مختلف داشته باشیم که همزمان از یک دیتای خام استفاده کنند (یکی برای آپدیت قیمت در دیتابیس، یکی برای ارسال نوتیفیکیشن هشدار قیمت).




PostgreSQL

استراتژی: در مایکروسرویس‌ها، ما بر اساس نیاز به خواندن (Read-heavy) یا نیاز به نوشتن (Write-heavy) تصمیم می‌گیریم:

- **Normalization (نرمال‌سازی در سرویس پایه):** در دیتابیسی که داده‌های پایه (سطح ۰ و ۱) را نگهداری می‌کند، کاملاً نرمال‌سازی می‌کنیم (3NF) تا از افزونگی جلوگیری کنیم. مثلاً نام فرودگاه فقط در جدول Airports ذخیره می‌شود و جدول پروازها فقط AirportID را نگه می‌دارد.

- **Denormalization (دی‌نرمال‌سازی در سرویس Aggregator):** همانطور که در نمودار معماری قبلی دیدیم، موتور جستجوی ترب باید در ۲۰ میلی‌ثانیه جواب بدهد. ما نمی‌توانیم در زمان سرچ، بین جدول‌های «ایرلاین»، «فرودگاه» و «قیمت» JOIN بزنیم! بنابراین، داده‌ها را در لایه جستجو (مثل Redis یا ElasticSearch) کاملاً دی‌نرمال (Denormalized) می‌کنیم. یعنی یک فایل JSON بزرگ (Document) می‌سازیم که تمام اطلاعات (نام آژانس، نام فرودگاه، قیمت) را در خود دارد.

چرا PostgreSQL برنده این سناریو است؟

- **پشتیبانی بی‌نظیر از JSONB:** ما دیتاهای قیمت را از آژانس‌های مختلف می‌گیریم که ممکن است فیلدهای متفاوتی داشته باشند (یک آژانس «مقدار بار مجاز» را می‌فرستد، دیگری نمی‌فرستد). PostgreSQL با نوع داده JSONB به ما اجازه می‌دهد داده‌های بدون ساختار مشخص را در کنار داده‌های رابطه‌ای ذخیره کنیم و روی آن‌ها با سرعت بالا (با Indexing) کوئری بزنیم. MariaDB در زمینه JSON ضعیف‌تر عمل می‌کند.

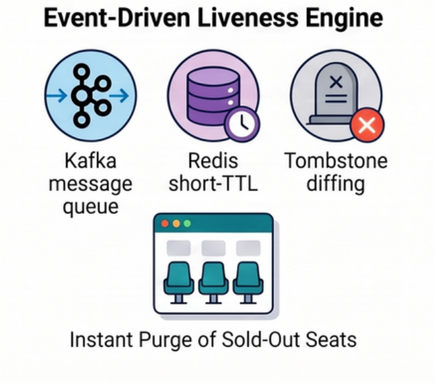
- **Concurrency و MVCC (مدیریت همروندی):** وقتی Scraperها در حال وارد کردن ده‌ها هزار ردیف قیمت جدید (Insert/Update) به جدول TicketOffer هستند، MariaDB (بسته به Storage Engine) ممکن است دچار قفل شدن (Locking) جداول شود و سرعت خواندن افت کند. معماری MVCC در PostgreSQL به شدت در محیط‌های Write-heavy (که همزمان خواندن‌های زیادی هم دارند) پایدارتر است.

- **Extensions (مانند PostGIS):** اگر در آینده ترب بخواهد ویژگی «پیدا کردن پرواز از فرودگاه‌های اطراف» (Location-based Search) را اضافه کند، اکستنشن PostGIS در PostgreSQL قدرتمندترین ابزار برای محاسبات جغرافیایی است.

- **شبیه‌سازی Time-Series (سری زمانی):** برای ذخیره تاریخچه قیمت‌ها (جهت رسم نمودار نوسان قیمت)، ما با داده‌های سری زمانی مواجهیم. PostgreSQL اکستنشن‌های قدرتمندی مثل TimescaleDB دارد که مدیریت میلیاردها رکورد قیمتی را در طول زمان بسیار بهینه می‌کند.

۱. لایه Ingestion (دریافت داده): سرویس Scraper: این سرویس‌ها مسئولیت فراخوانی API آژانس‌ها را دارند. **Topic کافکا: raw_flight_prices**
Scraperها فقط دیتای خام را در این Topic پرتاب می‌کنند. این کار باعث می‌شود اگر سرورهای پردازش ما داون شدند، دیتای گرفته‌شده از دست نرود.

۲. لایه پردازش (Processing) سرویس Normalizer: این سرویس (که در بخش قبل اسکریپت آن را نوشتیم) دیتای خام را از کافکا می‌خواند، نام ایرلاین‌ها و ساختارها را یکسان می‌کند و شناسه واحد (مثلاً THR-MHD-IR600-20261015) می‌سازد. **Topic کافکا: normalized_flights** دیتای تمیز در این Topic قرار می‌گیرد. حالا معماری ما Event-Driven (مبتنی بر رویداد) است. هر سرویس دیگری در سازمان می‌تواند به این Topic گوش دهد (Subscribe).



۳. لایه ذخیره‌سازی و دیتابیس (The DB Strategy)
برای بلیت هواپیما، ما با دو نوع نیاز مواجهیم: سرعت در حد میلی‌ثانیه برای جستجو، و ذخیره تاریخچه برای تحلیل.

- **Redis (Hot Storage) - برای Liveness**
  - **نقش:** این قلب تپنده جستجو است. وقتی کاربر در اپلیکیشن سرچ می‌کند، ما هرگز به دیتابیس رابطه‌ای (SQL) نمی‌رویم.
  - **ساختار:** دیتا به صورت Key-Value ذخیره می‌شود. (کلید: THR-MHD-20261015، مقدار: آرایه‌ای از پروازهای مرتب‌شده).
  - **Liveness:** کلید جادویی اینجا TTL (Time to Live) است. هر رکورد در ردیس ۱۵ دقیقه اعتبار دارد. اگر کاربر سرچ کرد و دیتا در ردیس نبود (Cache Miss)، API Gateway بلافاصله به سرویس Scraper فرمان می‌دهد که «همین الان دیتای تهران-مشهد را بگیر».
- **PostgreSQL (Cold Storage)**
  - **نقش:** اینجا قبرستان دیتا و معدن طلای تیم Data Science است.
  - **ساختار:** تغییرات قیمت‌ها در اینجا لاگ می‌شود. از این دیتا برای ماشین لرنینگ، نمودارهای پیش‌بینی قیمت (Predictive Alerts) و تحلیل رفتار رقبا استفاده می‌شود.

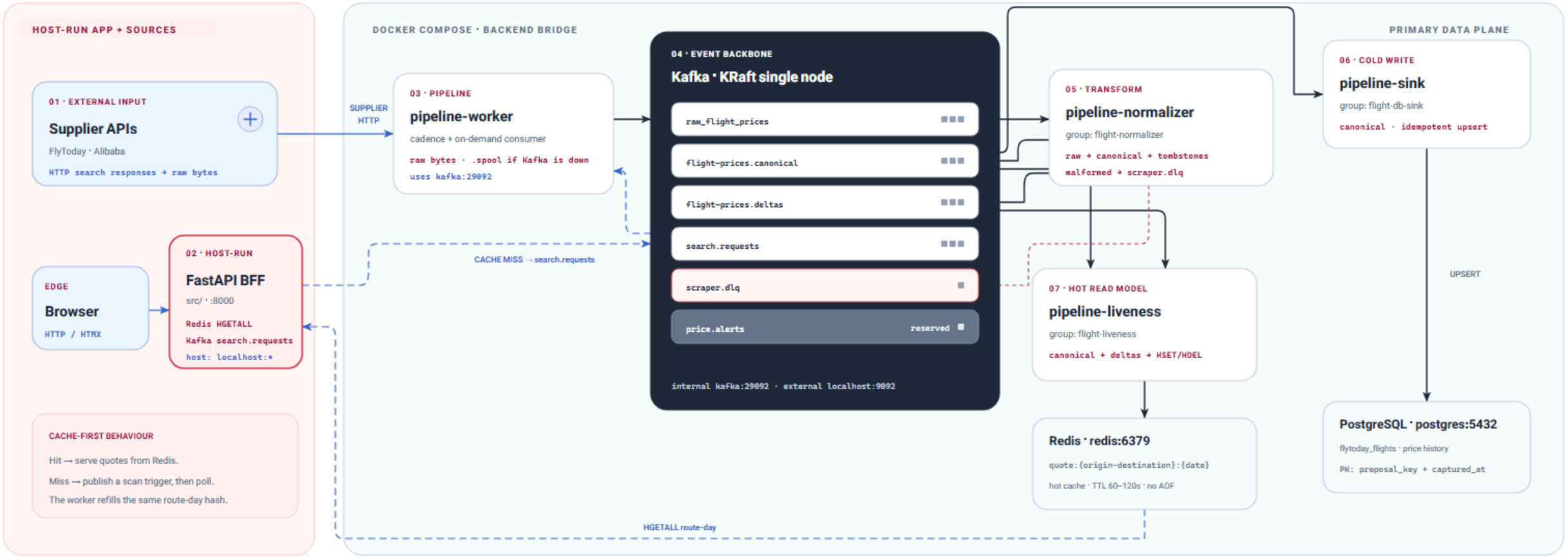


۴. سرویس‌های جانبی (Side-cars): Price Alert Service: این سرویس همزمان به Topic normalized_flights در کافکا گوش می‌دهد. به محض اینکه یک قیمت جدید وارد شد، چک می‌کند آیا کاربری برای این مسیر هشدار تنظیم کرده است؟ اگر بله، سریعاً نوتیفیکیشن می‌فرستد. این کار بدون اینکه باری روی سرور اصلی جستجو وارد کند انجام می‌شود.

این معماری، دقیقاً همان مدلی است که لیدرهای این صنعت (مانند Skyscanner) برای حفظ همزمان موجودی زنده و پرفورمنس بالا استفاده می‌کنند.

Kafka carries truth; Redis keeps the flight UI hot.

Verified against docker-compose.yml, docker-compose.pipeline.yml, pipelines/* and src/readmodel.py · base data plane + opt-in pipeline overlay



COMPOSE ADMIN / PROVISIONING · SIDE SERVICES, NOT THE DATA PLANE

kafka-init · creates six topics after health	kafka-ui :8080 · inspect Kafka	redisinsight :5540 · inspect Redis	pgadmin :5050 · inspect Postgres	all published ports bind to 127.0.0.1

Solid = data · dashed blue = cache-miss / read path · dotted red = malformed payloads / DLQ ■■■ = topic partitions

BFF is host-run · overlay: docker compose -f docker-compose.yml -f docker-compose.pipeline.yml up -d --build



docker



سه قطعه بسیار مهم و حیاتی پازل جا مانده است که اگر به آن‌ها فکر نکنیم، «ترب‌سفر» در همان ماه اول شکست می‌خورد:

۱. نقطه کور اول: فاجعه‌ی «Caching Cancellation» (لبه‌ی تاریک Liveness)

ما در معماری فنی (بخش کافکا و ردیس) روی آپدیت بودنِ قیمت‌ها تمرکز کردیم، اما ماهیت بلیت هواپیما فقط قیمت نیست، «موجودی صندلی» است.

- **مسئله جا مانده:** آژانس‌ها (مثل علی‌بابا) وقتی ظرفیت یک پرواز چارتری به صفر می‌رسد، آن را از API خود حذف می‌کنند (دیگر برنمی‌گردانند). اما سیستم ردیسِ ما (که دارای TTL مثلاً ۱۵ دقیقه‌ای است)، هنوز آن پرواز را به عنوان یک رکورد موجود به کاربر نشان می‌دهد، چون نمی‌داند که آژانس آن را حذف کرده است!
- **فاجعه‌ی UX:** کاربر پرواز ۲ میلیونی را می‌بیند، کلیک می‌کند تا از فلای‌تودی بخرد، اما در سایت مقصد با ارور «پرواز ناموجود است» مواجه می‌شود. نرخ خروج (Bounce Rate) و بی‌اعتمادی به ترب به شدت بالا می‌رود.
- **راه‌حل فنی جا مانده (Tombstoning):** ما باید در معماری، یک مکانیزم Soft Delete یا رویداد Sold Out Event طراحی کنیم. اسکریپت Scraper باید نه تنها پروازهای جدید را بگیرد، بلکه پروازهایی که در گرفتنِ قبلی بوده‌اند و الان نیستند را شناسایی کرده و یک سیگنال «اتمام موجودی» به کافکا بفرستد تا بلافاصله آن پرواز از کش (Redis) حذف شود یا لیبل قرمزِ «ظرفیت تکمیل» بخورد.

۲. نقطه کور دوم: استراتژی «Cold Start» در مارکتینگ (تله‌ی مرغ و تخم‌مرغ)

ما ساختار را برای هندل کردن کاربران طراحی کردیم، اما چرا کاربران باید روز اول برای خرید بلیت به ترب بیایند؟

- **مسئله جا مانده:** ترب در حال حاضر پلتفرمی برای "خرید کالا" است. ذهنیت کاربر (Mental Model) روی گوشی و قابلمه تنظیم شده، نه بلیت هواپیما. اگر یک تب جدید بالا اضافه کنیم، تضمینی نیست که کاربران متوجه آن بشوند یا به آن اعتماد کنند.
- **فاجعه‌ی پروداکتی:** محصول شیپ می‌شود، اما Adoption Rate (نرخ پذیرش) زیر ۱ درصد می‌ماند.
- **راه‌حل پروداکتی (Contextual In-App Cross-Selling):** نباید منتظر بمانیم تا کاربر تب «سفر» را کلیک کند. باید او را در مسیر خریدهای کالایی شکار کنیم!
  - **مثال:** اگر کاربری در حال جستجوی «چمدان»، «مایو شنا»، «کرم ضدآفتاب»، یا «چادر مسافرتی» است، دقیقاً زیر نتایج جستجوی کالا، یک بنر کوچک و هوشمند (Contextual Banner) از ترب‌سفر نشان دهیم: "آماده سفر می‌شی؟ ارزان‌ترین بلیت‌های این هفته را اینجا ببین." این کار بدون خرج کردن یک ریال بودجه مارکتینگ، کاربرِ دارایِ قصدِ سفر (Travel Intent) را مستقیماً به محصول جدید هدایت می‌کند.

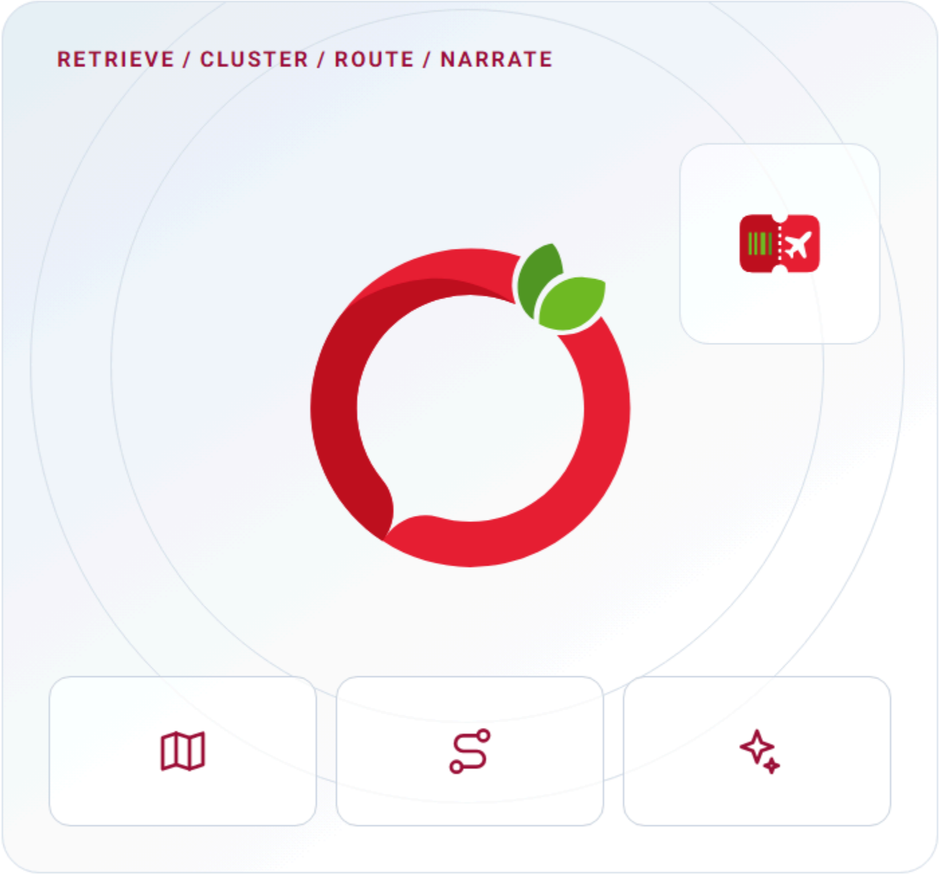
۳. نقطه کور سوم: چالش «Ghost Agencies» و کلاهبرداری (Trust & Verification)

ما در بخش کالا، سیستم «ضمانت ترب» و «امتیازدهی» را داریم، اما در بلیت این مکانیزم به کلی متفاوت است.

- **مسئله جا مانده:** وقتی شما مدل متاسرچ باز می‌کنید، ده‌ها آژانسِ بدون‌نام و نشان (Ghost Agencies) با استفاده از API چارترکننده‌ها، سایت‌های فروش بلیت می‌زنند. آن‌ها برای جذب مشتری از ترب، قیمت بلیت را به عمد ۵۰ هزار تومان زیرِ قیمت واقعی می‌زنند (کفِ لیست قرار می‌گیرند)، اما بعد از پرداخت توسط کاربر، بلیت صادر نمی‌شود یا پشتیبانی پاسخگو نیست.
- **فاجعه‌ی پروداکتی:** ترب مقصر شناخته می‌شود. کاربر می‌گوید "من از داخل ترب خریدم و سرم کلاه رفت".
- **راه‌حل پروداکتی (The Trust Gateway):**
  - **الف) White-listing دقیق:** ترب نباید مثل کالا، اجازه دهد هر سایت دارای اینماد به عنوان فروشنده بلیت ثبت‌نام کند. باید فقط آژانس‌های دارای مجوز بند (الف) از سازمان هواپیمایی کشوری اجازه اتصال داشته باشند.
  - **ب) شاخص اطمینان صدور (Fulfillment Score):** الگوریتم رنکینگ ترب در سفر نباید فقط بر اساس قیمت باشد. فرمول باید ترکیبی باشد از: (کف قیمت) + (نرخ موفقیت صدور بلیت آژانس) + (میانگین زمان پاسخگویی). اگر آژانسی ارزان‌ترین است اما از هر ۱۰ بلیت، ۲ تا را باطل می‌کند، باید اتوماتیک از رتبه یک به رتبه پایین‌تر سقوط کند.

TOROB / FEATURE DEEP-DIVE

From ticket to trip



The AI Itinerary Planner — after the cheapest ticket, Torob plans what happens next: an optimized route on an interactive map, a Persian timeline, an AI-written story for every day, and the lowest activity prices — in one page load.

Kish pilot · live today

The wedge: one island · ~500 places · then the playbook repeats

اضافه‌کردن یک «برنامه‌ریز هوشمند سفر» (AI Itinerary Planner) دقیقاً همان حلقه‌ی مفقوده‌ای است که ترب را از یک «ابزار مقایسه قیمت بلیت» به یک **«همراه سفر» (Travel Companion)** تبدیل می‌کند.

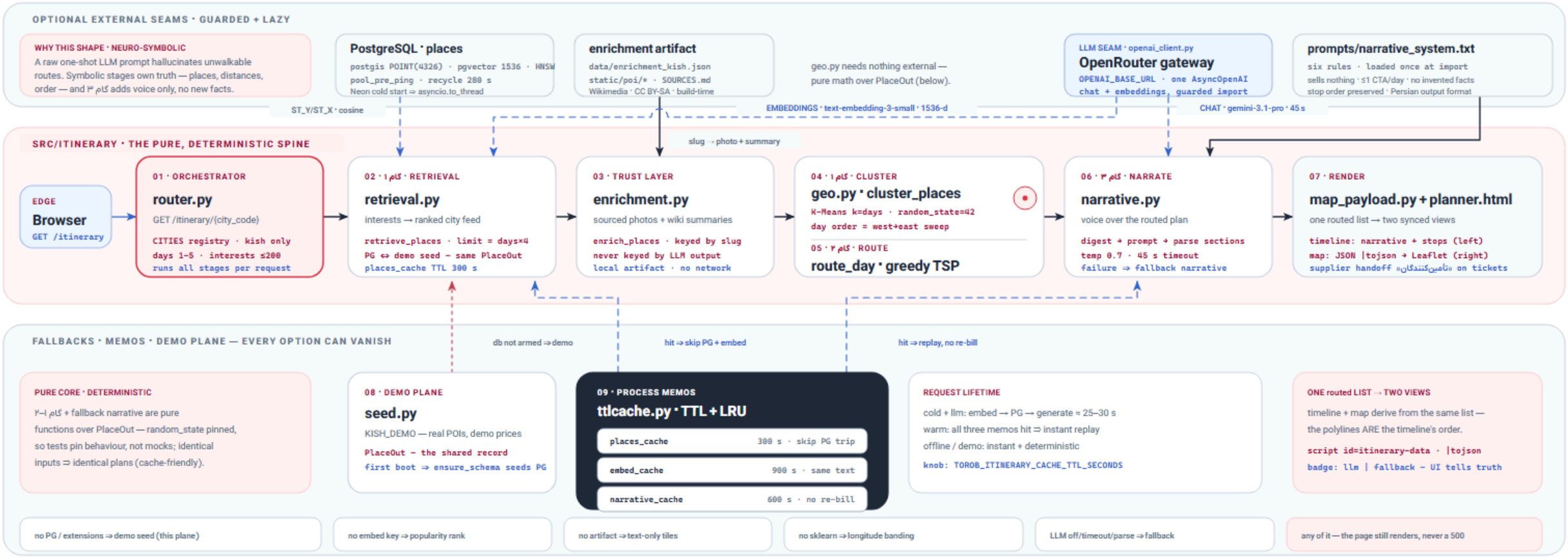
از دیدگاه یک معمار سیستم و مهندس محصول،
پیاده‌سازی این فیچر به یک تغییر معماری از دیتابیس‌های سنتی
به سمت پایگاه‌داده‌های مکانی (Spatial) و برداری (Vector) نیاز دارد.

برای ساخت AI Itinerary Planner (برنامه‌ریز هوشمند سفر)،
بزرگترین اشتباه مهندسان این است که کل کار را به یک LLM (مثل ChatGPT) می‌سپارند.
هوش مصنوعی در نقشه و ریاضیات (پیدا کردن نزدیک‌ترین مسیر و ساعات کاری) به شدت توهم (Hallucination) دارد.

معماری اصولی (Neuro-Symbolic AI):
ما ابتدا با ریاضیات و دیتابیس (بخش Symbolic) مکان‌ها را فیلتر، خوشه‌بندی و مسیریابی می‌کنیم،
سپس دیتای ریاضیِ آماده را به LLM (بخش Neuro) می‌دهیم تا برای آن یک «قصه و لحن جذاب» به همراه دکمه‌های فروش تور بسازد.

The symbolic stages own truth; the LLM only adds voice.

One GET /itinerary/{city_code}?days=…&interests=… request runs the whole pipeline — retrieve → enrich → cluster → route → narrate → render · every optional collaborator degrades, never a 500



AROUND THE PLANNER · TESTS · TOOLING · EXPANSION

- `tests/web/test_itinerary.py` — behaviour, not mocks
- `scripts/enrich_poi_media.py` — builds the artifact
- `static/poi/SOURCES.md` — provenance
- city #2 ⇒ a cities table (registry-not-table)
- `OPENAI_BASE_URL` ⇒ any OpenAI-compatible gateway

Solid ink = in-process · dashed blue = optional external seam / cache read · dotted red = fallback path — every path still renders a plan



docker



تا اینجای کار ما هسته یک پلتفرم سفر بسیار قدرتمند (موتور جستجوی بلیت + برنامه‌ریز هوشمند) را معماری کردیم. اما برای تبدیل شدن به یک سوپراپ سفر و ایجاد یک خندق رقابتی (Moat) عمیق که رقبا حتی با بودجه‌های مارکتینگ سنگین نتوانند از آن عبور کنند، باید به سمت فیچرهای نسل بعدی (Next-Gen) حرکت کنیم. **در ادامه، نقشه راه (Roadmap) استراتژیک برای توسعه فازهای بعدی «ترب‌سفر» را کالبدشکافی می‌کنیم:**

۱. بسته‌بندی پویا و متقاطع (Cross-Vendor Dynamic Bundling): آژانس‌هایی مثل علی‌بابا تورهای «پرواز + هتل» می‌فروشند، اما کاربر محدود به پروازها و هتل‌هایی است که خود علی‌بابا تامین کرده است.

- **دیدگاه پروداکتی (The Value):** ترب به عنوان متاسرچ می‌تواند به کاربر بگوید: «اگر پرواز رفت را از فلای‌تودی بخری، پرواز برگشت را از مستربلیت، و هتل را از جاباما رزرو کنی، پکیج سفرت ۳۰٪ ارزان‌تر از تورِ آماده‌ی علی‌بابا در می‌آید». این یعنی خلق یک تور اختصاصی (Custom Package) با پایین‌ترین قیمت ممکن در کل وبِ ایران.
- **معماری فنی:** محاسبات گراف دوبخشی (Bipartite Graph Matching): این یک مسئله بهینه‌سازی ترکیبیاتی (Combinatorial Optimization) است. بک‌اند باید در لحظه قیمت‌های پرواز (موجود در ردیس) را با قیمت‌های هتل (که باید از API اقامتگاه‌ها اسکرپ شود) کراس‌جوین (Cross-Join) کند.
- **چالش:** محاسبه ضرب دکارتیِ هزاران پرواز در صدها هتل، CPU-bound است. راهکار استفاده از موتورهای جستجوی In-memory مثل Elasticsearch یا نوشتن توابع Aggregation پیشرفته در PostgreSQL است تا در کسری از ثانیه پکیج‌های بهینه را فیلتر کنند.

۲. حالتِ کشف معکوس (Budget-Driven "Explore" Mode): کاربران همیشه نمی‌دانند کجا می‌خواهند بروند. گاهی فقط یک بودجه مشخص دارند (مثلاً ۵ میلیون تومان) و یک تعطیلات در پیش است.

- **دیدگاه پروداکتی:** یک اسلایدر بودجه در صفحه اول. کاربر می‌گوید «۵ میلیون تومان دارم». روی نقشه ترب (که با فیچر AI Planner ساختیم)، شهرهایی که با این بودجه می‌توان به آن‌ها پرواز کرد روشن می‌شوند.
- **معماری فنی:** ما در مستندات مهندسی معکوس دیدیم که فلای‌تودی و علی‌بابا APIهایی برای قیمت‌های تقویمی و پرتقاضا دارند (مثل SearchPopularPrices در فلای‌تودی یا cheapest در علی‌بابا).

به جای سرچ لحظه‌ای، یک Worker شبانه این APIهای بالک (Bulk) را می‌خواند و یک Inverted Index (شاخص معکوس) در دیتابیس می‌سازد: Budget -> Destination.
وقتی کاربر اسلایدر را می‌کشد، فرانت‌اند با HTMX و یک کوئری فوق‌سریع به ردیس، آرایه‌ای از مختصات (Lat/Lng) شهرها و کف قیمتشان را می‌گیرد و روی نقشه رندر می‌کند.

۳. برنامه‌ریزی مشارکتی و دنگ‌پرداز (Multiplayer Itinerary & Split-Pay): سفر معمولاً گروهی است. وقتی کاربر در ترب یک برنامه سفر (Itinerary) برای کیش می‌سازد، باید بتواند آن را با ۳ دوست دیگرش به اشتراک بگذارد تا با هم تصمیم بگیرند.

- **دیدگاه پروداکتی (Viral Loop):** کاربر لینک نقشه سفر را به دوستانش می‌فرستد. آن‌ها وارد ترب می‌شوند (جذب یوزر جدید و رایگان). دوستان روی رستوران‌ها یا کلوب‌ها رأی می‌دهند (Upvote/Downvote). در نهایت وقتی زمان خرید بلیت پرواز ۴ نفره یا رزرو ویلا می‌شود، سرویس «ترب‌پی» به صورت خودکار هزینه را بین ۴ کیف پول مختلف تقسیم می‌کند (دنگ الکترونیک). این ویژگی، ترب را از یک ابزار سرچ، به یک شبکه اجتماعیِ تراکنشی تبدیل می‌کند.
- **معماری فنی:** همگام‌سازی لحظه‌ای (Real-time Sync): برای اینکه کاربران رأی دادنِ همدیگر را روی نقشه ببینند، به جای REST API، از Server-Sent Events (SSE) یا WebSockets استفاده می‌کنیم. HTMX از طریق افزونه "hx-ext="sse به راحتی از این پروتکل پشتیبانی می‌کند.
- **مدیریت State:** استفاده از ساختار داده‌های CRDT (Conflict-free Replicated Data Types) در ردیس برای جلوگیری از تداخل ویرایش‌ها وقتی دو نفر همزمان یک برنامه سفر را تغییر می‌دهند.

۴. شکارچی هوشمند تاخیرها (Disruption & Refund Hunter): آژانس‌های مسافرتی پولی که بابت کنسلی یا تاخیر بالای ۲ ساعت پرواز به مسافر تعلق می‌گیرد را به سختی پس می‌دهند و کاربر معمولاً از حقوق خود بی‌خبر است.
دیدگاه پروداکتی: کاربر شماره پروازش را در ترب ثبت می‌کند. ترب به عنوان یک سرویس پس از فروش، بورد پروازهای فرودگاه (Flight Status API) را مانیتور می‌کند. اگر پرواز ۲ ساعت تاخیر خورد، ترب بلافاصله یک پوش نوتیفیکیشن می‌فرستد: "پرواز شما تاخیر داشت. طبق قانون، شما مستحق دریافت X٪ خسارت هستید. آیا می‌خواهید تیم حقوقی ترب این پول را از آژانس برای شما زنده کند؟" (Monetization: کسر کارمزد از پول زنده شده).

- **معماری فنی:** خرید و اتصال به API بورد اطلاعات فرودگاه‌های کشور (FIDS).

یک کافکا تاپیک مجزا (flight_status_events) که با دیتابیس Tickets جوین می‌شود. یک Rule Engine ساده که اگر Actual_Departure_Time - Scheduled_Time > 2 hours بود، یک ایونت به سرویس Notification بفرستد.

ترکیب این فیچرها، معماری ما را از یک لوله انتقال داده (Data Pipeline) یک‌طرفه، به یک اکوسیستم داده‌محور ارتقا می‌دهد؛ جایی که هوش مصنوعی مسیر را می‌چیند، موتور جستجو ارزان‌ترین تکه‌ها را به هم می‌چسباند، و ابزارهای فین‌تک (ترب‌پی) پرداخت‌های گروهی و قسطی را هندل می‌کنند. این اکوسیستم، استانداردی را تعریف می‌کند که رقبا با زیرساخت‌های مونولیتیک و سنتی خود، ماه‌ها (یا سال‌ها) زمان برای کپی‌برداری از آن نیاز خواهند داشت.


GitHub

THANK YOU VERY MUCH

Presented by
MM Khojasteh